

Умный Квартал

Дополнение: регистрация домов, ADMIN_USER_IDS, миграции и сценарии работы.

Обычное подключение дома происходит через заявку и одобрение; `/app provision` — инструмент оператора.

Развёртывание на удалённом сервере

Ubuntu 24.04 LTS | Docker Compose | HTTPS | MAX

Практическое руководство для SmartQuarter. Обновлено 26 сентября 2026 года: назначение председателей, регистрация домов, опросы, календарь и инициативы.

Что получится: сервер с HTTPS-входом, React Mini App, четырьмя Go-сервисами, тремя отдельными PostgreSQL и Redis. Фотографии хранятся во внешнем Yandex Object Storage. Команды рассчитаны на один VPS и небольшой пилотный запуск, без высокой доступности.

Интеграция: Gateway подключён к настоящему Identity по общему gRPC-контракту. Вход MAX создаёт сессию, доступ к дому определяется ACTIVE membership. Администратор назначает председателей в Mini App; председатель принимает заявки жильцов. Перед приёмкой нужны ваши домен, токен MAX, S3 и проверенные MAX ID.

В CI найдены и исправлены недостающие go.sum, незавершённые merge-конфликты, ошибки go vet и устаревшие тесты Identity. Новый pipeline проверяет также frontend и Docker-образы. Исправления локальные: для запуска GitHub Actions их нужно опубликовать отдельной веткой/PR.

Порядок работы: подготовка VPS → публикация исправлений → окружение и S3 → запуск сервисов → HTTPS и MAX → проверка готовности → обновления и резервное копирование.

В комплекте: `deploy/server/compose.yaml`, `Caddyfile`, `.env.example`, `s3-cors.json`. Не используйте старый `deploy/docker-compose.yml` как production-рецепт: в нём неполная конфигурация.

1. Что подготовить заранее

Рабочая отправная точка для пилота: Ubuntu 24.04 x86_64, 4 vCPU, 8 ГБ RAM, SSD от 40 ГБ. Это ориентир для сборки и нескольких сервисов, а не измеренный предел нагрузки. Для небольшого сервера лучше собирать образы в CI и доставлять из registry; текущий workflow пока не публикует их.

Нужны: SSH-доступ с sudo, домен, доступ к DNS, зарегистрированный бот MAX с токеном, приватный S3 bucket и ключ сервисного аккаунта. В примерах app.example.com заменяйте на свой домен. Внешний IP VPS подставляется в А-запись домена; AAAA создавайте только при настроенном IPv6.

Компонент	Внутренний адрес	Публичный доступ
Caddy	80 / 443	Да, единственная точка входа
Mini App	miniapp:8080	Только через Caddy
Gateway	max-gateway:8080	API и webhook через Caddy
Identity	identity-service:50051 / 8081	Нет
Issue	issue-service:8082 / 8083	Нет
Community	community-service:9090 / 8084	Нет
PostgreSQL × 3	отдельные сервисы, 5432	Нет
Redis	redis:6379	Нет
Object Storage	HTTPS endpoint провайдера	Только подписанные объекты

Схема: браузер MAX → Caddy → Mini App или Gateway → внутренние gRPC-сервисы. Загрузка фото: браузер → подписанный HTTPS PUT в S3. Базы и Redis не публикуются через ports. Общий Origin для UI/API упрощает cookie и CORS.

Внешний firewall провайдера: SSH только с вашего адреса, TCP 80/443 для сайта; UDP 443 необязателен для HTTP/3. Docker-публикация портов может обходить UFW: не добавляйте ports к БД и Redis, даже если UFW включён. Источник: Docker [1].

2. Подготовить Ubuntu и Docker

Команды этого раздела выполняются по SSH на новой Ubuntu. Сначала убедитесь, что SSH-доступ работает. Не отключайте текущий сеанс до проверки нового. Если используется нестандартный SSH-порт, разрешите именно его.

```
ssh deploy@SERVER_IP
sudo apt update
sudo apt install -y ca-certificates curl git openssl nano ufw
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 443/udp
sudo ufw enable
```

Установите Docker из официального apt-репозитория [1]. На сервере с уже установленным Docker сначала проверьте его состояние и существующие контейнеры; не удаляйте их ради повторной установки.

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo docker run --rm hello-world
sudo docker compose version
```

Ожидается успешный hello-world и вывод версии Compose. Далее команды Docker используют sudo. Добавление пользователя в группу docker не обязательно; такая группа практически эквивалентна root-доступу.

3. Доставить правильную версию проекта

Проверьте изменения и тесты, затем опубликуйте commit в ветке feat/house-identity-service. Для обычной установки после слияния используйте main. Сервер должен получать именно проверенный commit; локальные незакоммиченные файлы через git pull не передаются.

```
git status
git diff --check
git diff --stat
# После commit проверенных изменений:
git push -u origin feat/house-identity-service
```

На сервере создайте каталог приложения и клонируйте опубликованную ветку. Команда chown ниже рассчитана на обычного SSH-пользователя, а не root.

```
sudo mkdir -p /opt/smartquarter
sudo chown "$USER:$USER" /opt/smartquarter
git clone --branch main \
  https://github.com/Zhegly/SmartQuarter.git /opt/smartquarter
cd /opt/smartquarter
git status --short
git rev-parse HEAD
cd deploy/server
cp .env.example .env
chmod 600 .env
```

Если PR уже влит в main, используйте main и зафиксируйте выбранный SHA релиза. Для приватного репозитория понадобится read-only deploy key или подходящая авторизация Git. Не добавляйте PAT в URL команды.

С этого места рабочий каталог всех compose-команд: /opt/smartquarter/deploy/server. Compose читает .env из этого каталога. После выхода и повторного SSH вернитесь сюда; иначе можно случайно запустить другой проект.

4. Заполнить окружение

Откройте `.env` командой `nano .env`. Это серверный файл: не копируйте его в `web/miniapp` и не коммитьте. Три независимых пароля БД и `webhook secret` удобно получить командами `openssl rand -hex 32`, выполненными четыре раза. Нех не требует экранирования в PostgreSQL URL.

Переменная	Что указать
APP_DOMAIN	app.example.com, без https:// и пути
ACME_EMAIL	Адрес администратора для сертификатов
IDENTITY_DB_PASSWORD	Отдельный случайный hex-пароль
ISSUE_DB_PASSWORD	Другой случайный hex-пароль
COMMUNITY_DB_PASSWORD	Третий случайный hex-пароль
MAX_BOT_TOKEN	Настоящий токен бота, только на сервере
MAX_WEBHOOK_SECRET	Случайная строка A-Z, a-z, 0-9, _ или -
MAX_BOT_USERNAME	Имя бота без @; для кнопки open_app
MAX_CA_BUNDLE	Путь к доверенному CA bundle на сервере; см. раздел 8
ADMIN_USER_IDS	Внутренние UUID администраторов через запятую; см. раздел 9
S3_BUCKET	Имя существующего приватного bucket
AWS_ACCESS_KEY_ID	Статический ключ сервисного аккаунта
AWS_SECRET_ACCESS_KEY	Его секретная часть

Для выбранного в гайде Yandex Object Storage оставьте `S3_ENDPOINT` и `S3_PUBLIC_ENDPOINT` равными <https://storage.yandexcloud.net>, `S3_REGION=ru-central1`. В Compose включён `path-style`: bucket указывается в пути, а не в домене. При другом провайдере перепроверьте `endpoint`, регион, `path-style` и `VITE_S3_ORIGINS`.

```
nano .env
sudo docker compose config --quiet
```

Ожидается отсутствие ошибок. Пустые обязательные значения специально останавливают Compose. Не используйте обычный `docker compose config` в общих логах: он выводит уже предоставленные секреты.

`VITE_API_BASE_URL` в образе frontend пустой: запросы идут на тот же HTTPS Origin. `VITE_S3_ORIGINS` встраивается при сборке из `S3_PUBLIC_ENDPOINT`. Изменение этих публичных параметров требует пересборки Mini App; `docker restart` не меняет bundle. Серверные токены во frontend не передаются.

5. Создать и проверить Object Storage

В Yandex Cloud создайте bucket для фотографий. Оставьте публичное чтение, листинг и запись выключенными. Создайте сервисный аккаунт со статическим access key и правами чтения/записи в этот bucket. В .env внесите ключи и имя bucket. Не используйте ключ владельца всего облака, если можно ограничить доступ bucket. Хранилище оплачивается отдельно от VPS.

В CORS bucket разрешите точный Origin <https://app.example.com>, методы GET, HEAD, PUT и заголовки подписанного запроса. Шаблон deploy/server/s3-cors.json нужно изменить под реальный домен. CORS не делает bucket публичным и не заменяет подпись. Инструкция провайдера: [4].

Если установлен совместимый AWS CLI и задан профиль sq-storage со статическим ключом, конфигурацию можно применить так:

```
aws --profile sq-storage \
  --endpoint-url https://storage.yandexcloud.net \
  s3api put-bucket-cors --bucket YOUR_BUCKET \
  --cors-configuration \
  "$(jq '{CORSRules: .}' s3-cors.json)"
```

Эта необязательная команда требует aws и jq; можно выполнить ту же настройку через облачную консоль. Секреты профиля храните в закрытом файле credentials. Для неё замените YOUR_BUCKET, а не оставляйте буквальный пример.

Проверка из UI должна пройти всю цепочку:

- POST /api/v1/uploads возвращает upload_id, presigned_url и required_headers.
- Браузер отправляет сам File через PUT на HTTPS S3 URL, без cookie Gateway.
- POST /api/v1/uploads/{id}/complete возвращает READY.
- CreateIssue получает attachment_ids; затем фото доступно через download-url.

403 от S3 обычно означает ошибку подписи, изменённый Content-Type, просроченный URL или неверные права. Ошибка CORS видна в DevTools: проверьте Origin, OPTIONS/PUT и разрешённые headers. Адрес minio:9000 или localhost в presigned URL не будет доступен телефону. Подписанные URL не публикуйте в чатах и логах диагностики.

6. Собрать и запустить сервисы

Команды запускаются из `deploy/server`. На первой сборке Docker скачает образы и зависимости; оставьте достаточно места на диске и времени. Не используйте Node/Vite dev server для публичного production.

```
sudo docker compose config --quiet
sudo docker compose build
sudo docker compose up -d
sudo docker compose ps -a
sudo docker compose logs --tail=100 \
  identity-service issue-migrate community-migrate
```

Что происходит при запуске:

- PostgreSQL создаёт `identity_db`, `issue_db` и `community_db` в разных volumes.
- Identity самостоятельно выполняет встроенные Goose migrations.
- `issue-migrate` запускает `/app migrate up`; Issue ждёт успешного завершения.
- `community-migrate` выполняет SQL 000001, 000002 и 000003 через `psql` с `ON_ERROR_STOP`.
- Сервисы подключаются к единому Redis; уведомления используют `stream:notifications`.
- Caddy начинает выдавать сайт по HTTPS после корректной настройки DNS.

Одноразовые migration-контейнеры должны завершиться с кодом 0: статус `Exited (0)` для них нормален. `Exited (1)`, постоянный `Restarting` или нездоровая БД требуют проверки логов. Community сейчас не имеет полноценного версионированного migrator: начальный `CREATE TABLE IF NOT EXISTS` нельзя считать способом обновления будущих колонок.

PostgreSQL и Redis доступны только внутри Docker network. Не добавляйте публичные порты 5432, 6379, 50051, 8082 и 9090. Для пилота PostgreSQL image создаёт владельца каждой БД; для зрелого production разделите `migration/admin role` и `runtime role` с минимальными правами.

Готовность: Gateway `/readyz` проверяет Redis, настоящий Identity (включая БД), Issue и Community. Ответ 503 теперь означает недоступную зависимость. Проверьте адреса сервисов и их `/readyz`; успешный `/livez` проверяет только процесс. Вход нового пользователя без назначенного дома возвращает сессию и пустой активный дом, затем UI предлагает получить доступ.

7. HTTPS, маршруты и проверка сайта

В DNS установите A-запись `app.example.com` на публичный IPv4 VPS. Если есть AAAA, она также должна вести на этот сервер. Порты 80/443 должны доходить до Caddy. Проверьте, что их не занял другой nginx/apache. Caddy получает и продлевает сертификат автоматически; каталог `/data` сохранён в отдельном volume [2].

```
getent ahosts app.example.com
sudo ss -lntp | grep -E ':80|:443'
sudo docker compose logs --tail=100 caddy
curl -I https://app.example.com/
curl -I https://app.example.com/issues/invalid-test-id
curl -i https://app.example.com/api/v1/me
```

Главная и SPA route возвращают HTML. Некорректный UUID показывает ошибку в React, а не 404 статического сервера. GET `/api/v1/me` без cookie должен вернуть 401 JSON; это подтверждает, что запрос попал в Gateway. HTML вместо JSON означает ошибку reverse проху. 502 обычно означает, что upstream не запущен или указан неверный порт.

Caddyfile сохраняет исходный URI и направляет `/api/v1/*` и `/webhooks/max` на `max-gateway:8080`. Всё остальное идёт на `miniapp:8080`. Не заменяйте `handle` на `handle_path`: последний удалит префикс API. Заголовки Origin/Cookie должны доходить до Gateway; не подставляйте доверенный Origin вместо любого клиента.

Технические `/metrics` и backend `/readyz` не опубликованы наружу. Для внутренней диагностики используйте `wget` из контейнера Caddy:

```
sudo docker compose exec caddy \
  wget -qO- http://identity-service:8081/readyz
sudo docker compose exec caddy \
  wget -qO- http://issue-service:8083/readyz
sudo docker compose exec caddy \
  wget -qO- http://community-service:8084/readyz
sudo docker compose exec caddy \
  wget -S -O- http://max-gateway:8080/readyz
```

Все четыре проверки должны стать успешными при доступных зависимостях; Issue также проверяет S3. Простой HTTP 200 frontend не доказывает готовность backend.

8. Подключить бота и Mini App в MAX

В кабинете MAX создайте/выберите своего бота и настройте HTTPS URL мини-приложения <https://app.example.com>. Токен скопируйте в серверную .env. MAX_BOT_USERNAME должен соответствовать этому боту. Наличие кода в репозитории не означает, что бот уже зарегистрирован и webhook подписан.

Gateway обрабатывает bot_started, message_created с /start и message_callback. Он отправляет кнопку open_app; обычную HTML-страницу нельзя считать эквивалентом запуска через MAX Bridge. Подписка на webhook выполняется отдельно: POST /subscriptions, URL <https://app.example.com/webhooks/max>, secret совпадает с MAX_WEBHOOK_SECRET. Официальная документация: [3].

После запуска HTTPS можно зарегистрировать подписку из интерактивного Bash на доверенном компьютере. Команда меняет доставку событий выбранного бота; не используйте токен другого бота. Установите jq, если отсутствует. Ввод токена скрыт и не записывается как буквальный текст команды в history.

```
sudo apt install -y jq
read -rsp 'MAX bot token: ' BOT_TOKEN; printf '\n'
read -rsp 'Webhook secret from .env: ' WEBHOOK_SECRET; printf '\n'
read -rp 'App domain, without https://: ' APP_DOMAIN
PAYLOAD=$(jq -n \
  --arg url "https://${APP_DOMAIN}/webhooks/max" \
  --arg secret "$WEBHOOK_SECRET" \
  '{url:$url, secret:$secret,
    update_types:["bot_started","message_created","message_callback"]}')
curl --fail-with-body -sS \
  -X POST https://platform-api2.max.ru/subscriptions \
  -H "Authorization: ${BOT_TOKEN}" \
  -H 'Content-Type: application/json' \
  --data "$PAYLOAD"
unset BOT_TOKEN WEBHOOK_SECRET PAYLOAD
```

Проверьте не только HTTP 200, но и success:true в JSON. Затем отправьте /start своему боту и нажмите кнопку приложения. Не проверяйте webhook случайной настоящей командой вручную: это может отправить сообщение пользователю. Gateway проверяет X-Max-Bot-API-Secret, поэтому запрос без него закономерно даст 401.

При ошибке x509 у Bot API проверьте актуальную цепочку доверия, включая требования MAX к сертификату Минцифры [3]. Установите сертификаты из официального источника <https://www.gosuslugi.ru/landing/crt> в /usr/local/share/ca-certificates на Ubuntu и выполните sudo update-ca-certificates. В deploy/server/.env задайте MAX_CA_BUNDLE=/etc/ssl/certs/ca-certificates.crt. Compose монтирует этот файл в Gateway только для чтения; после изменения пути пересоздайте Gateway. Для scratch-образа пакетный менеджер внутри контейнера недоступен. Не используйте curl -k и не отключайте TLS-проверку в Go.

9. Создать дом и назначить роли

Identity уже подключён к Gateway через `contracts/proto/smartquarter/identity/v1/identity.proto`. `IDENTITY_GRPC_ADDR` направлен на `identity-service:50051`. Адаптер преобразует enum ролей, даты и MAX ID; каждый защищённый запрос заново проверяет `membership`. Отзыв роли действует и для ранее созданной сессии.

Новый пользователь входит через MAX без дома. Первый вход не выдаёт роль администратора. Оператор проверяет MAX ID будущего администратора и находит внутренний UUID:

```
sudo docker compose exec identity-db psql -U identity -d identity_db
```

```
SELECT id, max_user_id, display_name FROM users
WHERE max_user_id = 123456789;
```

Подставьте проверенный MAX ID вместо примера. В `deploy/server/.env` запишите полученный UUID в `ADMIN_USER_IDS`; несколько UUID разделяются запятыми. Примените конфигурацию:

```
sudo docker compose up -d --force-recreate identity-service
```

Администратор открывает «Профиль → Администрирование платформы», находит будущего председателя и нажимает «Назначить председателем». Пользователь должен хотя бы раз войти через MAX. Председатель подаёт заявку через «Дома → Зарегистрировать дом». Администратор рассматривает её в разделе регистраций. После одобрения создаются дом и активное председательство.

Жители находят дом через поиск или приглашение и отправляют заявки. Председатель принимает их в разделе заявок жильцов. Принятый житель выбирает дом и получает доступ к данным.

Для существующего дома администратор выбирает пользователя и назначает его в карточке дома. Прежний председатель становится жителем. «Снять с этого дома» снимает конкретное назначение. «Снять полномочия» снимает председательство во всех домах и отменяет ожидающие регистрации и передачи роли. Доступ жителя сохраняется. Роль ADMIN в `membership` относится только к дому и не даёт доступа к платформенной админке.

Передача полномочий доступна активному жителю, которого администратор уже назначил кандидатом в председатели. Получатель должен подтвердить передачу в течение 48 часов. Команда `/app provision` остаётся закрытым инструментом оператора для восстановления доступа; обычное подключение выполняется через интерфейс.

10. Приёмка после запуска

Проверки разделены на инфраструктуру и продукт: это позволяет понять, на каком шаге произошла ошибка. Вход и бизнес-сценарий требуют реальных настроек MAX/S3 и назначенных ролей из раздела 9.

Уровень	Ожидаемый результат
DNS и TLS	Домен ведёт на VPS; сертификат доверенный
Frontend	/ и вложенный route открывают Mini App
Proxy	/api/v1/me без cookie даёт 401 JSON
Базы/схемы	3 БД healthy; миграции успешны
Backend	/readyz Identity/Issue/Community = 200
Gateway	/readyz = 200 при здоровых зависимостях
MAX auth	Bootstrap = 200, HttpOnly cookie, правильный дом
S3	PUT/complete/просмотр фото успешны

Полный сценарий выполните пять раз на тестовых аккаунтах и выделенном тестовом доме:

1. Resident A открывает Mini App из MAX; видит своё имя, адрес и роль.
2. Создает проблему с JPEG/PNG: presign → PUT → complete READY → Issue.
3. Находит Issue в списке, открывает карточку и фото.
4. Resident B подтверждает его; счётчик обновляется, повторная кнопка отключена.
5. Chairman видит проблему в очереди, формирует заявление, проверяет body/version.
6. Копирует текст и меняет допустимый статус с подтверждением действия.
7. Публикует объявление; оба жителя видят его в ленте.
8. Переключение дома убирает прежние данные; resident не открывает chairman UI.

Проверьте также истечение сессии, другой дом, неверный Origin, S3 CORS, клавиатуру, BackButton и копирование на MAX Android/iOS. Скачивание .txt в нативном MAX сейчас ограничено отсутствием серверного HTTPS file endpoint; используйте копирование. Проверьте раздел «Сообщество»: создание и закрытие опроса, единственный голос жителя, результаты; создание, изменение и удаление события; публикацию инициативы, однократную поддержку и завершение председателем. На внутренних экранах должны оставаться кнопка «Назад» и нижняя навигация.

Дополнительно запустите из корня: `docker compose -f deploy/test/compose.yaml run --build --rm test`. Стенд проверяет production router Gateway, настоящие Identity/Issue/Community, PostgreSQL, Redis и MinIO; внешний MAX подменён тестовым HTTP-сервером. Браузерные fixtures и этот тест не подтверждают облачные права, публичный TLS и работу на устройствах MAX.

11. Обновление и откат

Перед обновлением сохраните текущий SHA, образы и backup БД. Identity автоматически применяет миграцию 000003_chairman_permissions; существующие активные председатели сохраняют возможность регистрации домов. Community требует запуска community-migrate, включая 000003_community_integrity. Если проверка связи голоса с вариантом не проходит, миграция остановится без удаления данных: выясните происхождение некорректных записей перед продолжением.

Перед технической сдачей выполните `python tools/data-api/validate_data_api.py DATA-API.yaml --openapi contracts/openapi/openapi.yaml` из корня. Файл содержит обязательные HTTP-проверки; сессии проверяющим передаются отдельно, по ролям.

Базы меняются независимо от приложения: возврат к старому image не отменяет миграции. Для breaking schema change нужен отдельный план совместимости или восстановление в новые volumes.

```
cd /opt/smartquarter
PREVIOUS_SHA=$(git rev-parse HEAD)
printf '%s\n' "$PREVIOUS_SHA" > /tmp/sq-previous-sha
# После review выберите SHA прошедшего CI, не произвольный latest.
git fetch origin
git switch --detach NEW_VERIFIED_COMMIT_SHA
cd deploy/server
sudo docker compose config --quiet
sudo docker compose build
# Контролируемое короткое окно обслуживания без удаления volumes.
sudo docker compose stop max-gateway issue-service community-service
sudo docker compose run --rm issue-migrate
sudo docker compose run --rm community-migrate
sudo docker compose up -d --force-recreate \
  identity-service issue-service community-service max-gateway miniapp caddy
sudo docker compose ps -a
```

Перед применением community-migrate просмотрите изменения SQL: текущий начальный скрипт не обновит уже существующую таблицу новым столбцом. Не меняйте POSTGRES_PASSWORD для существующего volume только в .env: пользователь БД не получит новый пароль автоматически. Ротация выполняется через ALTER ROLE и согласованное обновление приложений.

Простой откат кода допустим только при обратной совместимости схемы:

```
cd /opt/smartquarter
git switch --detach "$(cat /tmp/sq-previous-sha)"
cd deploy/server
sudo docker compose build
sudo docker compose up -d --force-recreate
```

Не используйте `docker compose down -v` для обычного обновления: это удалит данные. Не делайте автоматический migrate down для production без проверенного backup. После любого обновления заново выполните /readyz и короткий пользовательский smoke test.

Для воспроизводимых релизов следующий шаг - registry с образами, помеченными SHA/digest, и отдельное deployment approval. Нынешний CI только проверяет и собирает; автоматического SSH-деплойа в репозитории нет.

12. Резервное копирование и восстановление

Храните резервные копии вне VPS, с шифрованием и ограниченным доступом. `pg_dump` каждой БД обеспечивает её внутреннюю согласованность, но не общую транзакцию между тремя сервисами. Для пилота остановите бизнес-сервисы на время набора backup; Caddy и БД можно оставить запущенными.

```
cd /opt/smartquarter/deploy/server
(
set -euo pipefail
umask 077
BACKUP_DIR="$HOME/sq-backups/$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "$BACKUP_DIR"
trap 'sudo docker compose start identity-service issue-service \
community-service max-gateway' EXIT
sudo docker compose stop max-gateway identity-service \
issue-service community-service
for pair in identity:identity_db issue:issue_db community:community_db; do
user=${pair%:*}; db=${pair#*:}
sudo docker compose exec -T "$user-db" \
pg_dump -U "$user" -d "$db" -Fc > "$BACKUP_DIR/$db.dump"
done
cp .env "$BACKUP_DIR/server.env"
git -C ../../ rev-parse HEAD > "$BACKUP_DIR/release-sha.txt"
ls -lh "$BACKUP_DIR"
printf 'Backup complete: %s\n' "$BACKUP_DIR"
)
```

Подоболочка останавливается при ошибке `pg_dump`; `trap` запускает сервисы обратно и при успехе, и при ошибке. Отсутствие строки `Backup complete` означает незавершённый набор. Проверьте размеры и читаемость архивов; файл не становится валидным только потому, что существует. Для ежедневной автоматизации добавьте журнал и контроль восстановления.

Отдельно нужны: объекты S3 (versioning/backup-политика bucket), Redis AOF для потоков/сессий, Caddy /data с сертификатами. Дамп PostgreSQL не содержит фотографии. Не копируйте живой каталог PostgreSQL обычным `cp`. Redis snapshot/volume backup делайте согласованным способом; потеря сессий допускает повторный вход, но потеря Streams требует сверки outbox и уведомлений.

Проверяйте восстановление сначала в отдельном Compose project и чистых volumes, без публичного webhook. Пример команды для заранее созданной пустой тестовой `identity_db`:

```
# Подключаться только к изолированному recovery-проекту!
sudo docker compose -p sq-recovery exec -T identity-db \
pg_restore -U identity -d identity_db --exit-on-error \
< /PATH/TO/identity_db.dump
```

Не добавляйте `--clean` к восстановлению рабочей БД без отдельного плана. Восстановление должно включать сверку `house/user UUID`, вложений S3 и совместимой версии приложения. Backup без пробного `restore` не является доказанной возможностью восстановления.

13. Диагностика типичных ошибок

Симптом	Причина / следующий шаг
Сайт недоступен	Проверить A/AAAA, firewall провайдера, 80/443, логи Caddy
Ошибка сертификата	DNS/ACME/системное время; не отключать проверку TLS
API отвечает HTML	/api/v1 попал в SPA; проверить Caddy matcher без strip prefix
502 от проху	Контейнер не запущен, неправильное имя/порт, ошибка старта
Bootstrap 503	Недоступная зависимость; проверить Identity gRPC/БД и остальные /readyz
Bootstrap 401	Неверная/старая initData, токен другого бота; переоткрыть MAX
Mutation 403	TRUSTED_ORIGINS или membership/поль; не подменять headers
S3 PUT 403	Права, подпись, Content-Type, срок URL, часы сервера
CORS в браузере	Origin bucket и required_headers должны совпадать
Issue unhealthy	Проверить БД, миграции и доступ к bucket
Нет объявлений / 503	Community address, база/схема, логи сервиса
Страница просит MAX	Обычный браузер не предоставляет подписанную initData
Нет уведомлений	Identity mapping, Redis stream, токен/доставка Bot API
Старый frontend ENV	Пересобрать image; restart не меняет Vite bundle

```

sudo docker compose ps -a
sudo docker compose logs --tail=150 max-gateway
sudo docker compose logs --tail=150 issue-service
sudo docker compose logs --tail=150 identity-service
sudo docker compose logs --tail=150 community-service
sudo docker compose exec redis redis-cli PING
sudo docker compose exec issue-db \
  psql -U issue -d issue_db -c 'SELECT now();'
df -h
free -h
timedatectl status

```

Для обмена диагностикой сохраняйте request_id и HTTP status. Не передавайте .env, session cookie, raw initData, S3 signed URL или токен бота. Если они утекли, ротируйте соответствующий секрет и инвалидируйте сессии/подписки, а не просто удаляйте сообщение.

На Windows Docker Desktop может быть запущен как UI, но Linux Engine недоступен. В этом случае локальная ошибка named pipe не доказывает проблему Dockerfile. Сервер Ubuntu использует Docker Engine; проверьте `sudo systemctl status docker` и `sudo docker info`.

14. Что произошло с CI и что исправлено

Проверенный GitHub Actions run: 35665544780, main, commit 9c4e70d. Он завершился failure на шаге Format, vet and test Go modules, ещё до проверок frontend. Прямая ссылка: [5].

Первая ошибка в логах: отсутствующие go.mod checksums в community-service/go.sum (multierr, x/net, genproto/rpc, x/text, procfs). GOWORK=off в CI не использует go.work.sum, поэтому рабочая локальная workspace-сборка не гарантирует независимую сборку каждого сервиса.

При последовательной локальной проверке обнаружены и исправлены также:

- В Identity cmd/app/main.go и HTTP handler остались текстовые merge markers. Сохранён актуальный запуск через app.Run и новый NewRouter.
- Старые тесты Identity ссылались на удалённый NewHandler и /healthz. Проверки обновлены на /livez, /readyz с доступной/недоступной БД и неизвестный route.
- Community использовал status.Errorf(code, err.Error()). Заменено на status.Error без интерпретации сообщения как format string.
- В Issue main.go нарушен gofmt. Восстановлено форматирование.
- go.sum Community, Issue и Gateway дополнены через go mod tidy без обновления версий go.mod.
- Identity Dockerfile приведён к Go 1.26.5, как остальные основные Go-образы, вместо устаревшего build-stage 1.22.

Новый workflow: отдельные Go matrix jobs, проверка gofmt/tidy/vet/tests; Node 24, typecheck/lint/unit/build/Playwright; сборка пяти Docker images и отдельный контейнерный application workflow с реальным Identity. Итоговый check CI зависит от всех групп и не будет зелёным при failure или skipped dependency. Проверки не отключены ради зелёного статуса.

В этом репозитории был только CI. CD, публикации образов и автоматического применения на VPS не было; они не имитируются новым workflow. Для CD отдельно потребуются registry, SSH/deploy credentials, подтверждение production environment, backup и rollback-политика.

Исправления нужно закоммитить и отправить в ветку, открыть PR и дожждаться нового Actions run. Старый failed run останется красным - он относится к прежнему commit. Локальные результаты и ограничения проверки зафиксированы в docs/deployment/ci-audit.md.

15. Финальная памятка и источники

До открытия для жителей должны быть выполнены все условия:

- Опубликован выбранный SHA с исправлениями и новым серверным Compose.
- Новый CI прошёл на этом SHA, а не только на соседней ветке.
- .env закрыт, публичны только нужные входные порты, есть backup вне VPS.
- Домен/HTTPS, миграции и readiness всех сервисов проверены.
- Gateway /readyz = 200, bootstrap создаёт cookie и отдаёт назначенную роль.
- Настроены MAX Mini App URL, webhook secret и подписка на события.
- Есть проверенные пользователи, дом и ACTIVE memberships с нужными ролями.
- Выполнены реальные S3 PUT/complete и пять бизнес-сценариев.
- Проверено восстановление БД, объектов и конфигурации.

Этот документ описывает подготовку и порядок развёртывания. Успешные локальные тесты не подтверждают обновление конкретного VPS: после применения новой версии выполните приёмку на своём домене и внутри MAX по разделу 10.

Источники, проверенные 22.09.2026:

[1] Docker Engine, Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>

[2] Caddy, automatic HTTPS: <https://caddyserver.com/docs/automatic-https> ; маршрутизация: https://caddyserver.com/docs/caddyfile/directives/reverse_proxy

[3] MAX, webhook subscriptions: <https://dev.max.ru/docs-api/methods/POST/subscriptions> ; Mini Apps: <https://dev.max.ru/docs/webapps/introduction>

[4] Yandex Object Storage, CORS: <https://yandex.cloud/ru/docs/storage/operations/buckets/cors>

[5] Фактический CI run: <https://github.com/ZheglY/SmartQuarter/actions/runs/35665544780>

Исходники проекта: <https://github.com/ZheglY/SmartQuarter> ; точки проверки - .github/workflows/ci.yml, services/*/go.mod, Dockerfile, cmd/app, config, migrations, web/miniapp/README.md.

Сопутствующие файлы: deploy/server/compose.yaml, Caddyfile, .env.example, s3-cors.json; исходник этого гайда - docs/deployment/server-guide.md. При изменении контрактов или инфраструктуры сначала обновите текстовый runbook, затем пересоберите PDF.